Linux Memory: Virtual Memory, MMU & Page Tables

How the hardware MMU and Linux kernel map 4KB virtual pages to physical DRAM using multi-level page tables, TLB, and Swap.

1. Virtual Address Space & MMU

Every Linux process runs inside an isolated 128TB virtual address space. Programs never access physical RAM addresses directly; the CPU MMU translates addresses via the `CR3` register.

```
Virtual Address → CPU MMU → Translation Cache (TLB) → Physical RAM
```

2. 4-Level Page Tables Hierarchy

x86_64 uses multi-level paging so sparse memory allocations don't waste gigabytes of table overhead:

```
CR3 → PGD → P4D → PUD → PMD → PTE → Physical 4KB Frame
```

- TLB hit: 1 nanosecond lookup
- TLB miss: 4 memory access penalties

3. Page Faults & Demand Paging

Linux allocates memory lazily via `malloc()` / `brk()`. Physical RAM is allocated only when the page is first written to:

- **Minor Page Fault**: Page is in RAM, just map to page table
- **Major Page Fault**: Page swapped to disk, must read from SSD

4. Linux Memory Inspection Commands

Professional system diagnostics:

```
# View dirty pages, slab cache & free RAM
cat /proc/meminfo | grep -E 'MemAvailable|Dirty|Slab'

# Live page-in / page-out rate
vmstat -s -S M

# Swap partition usage breakdown
swapon --show
```